

How Variables Live in Memory

A Deep Dive into Value, Reference, and
the Special Case of Strings in Node.js

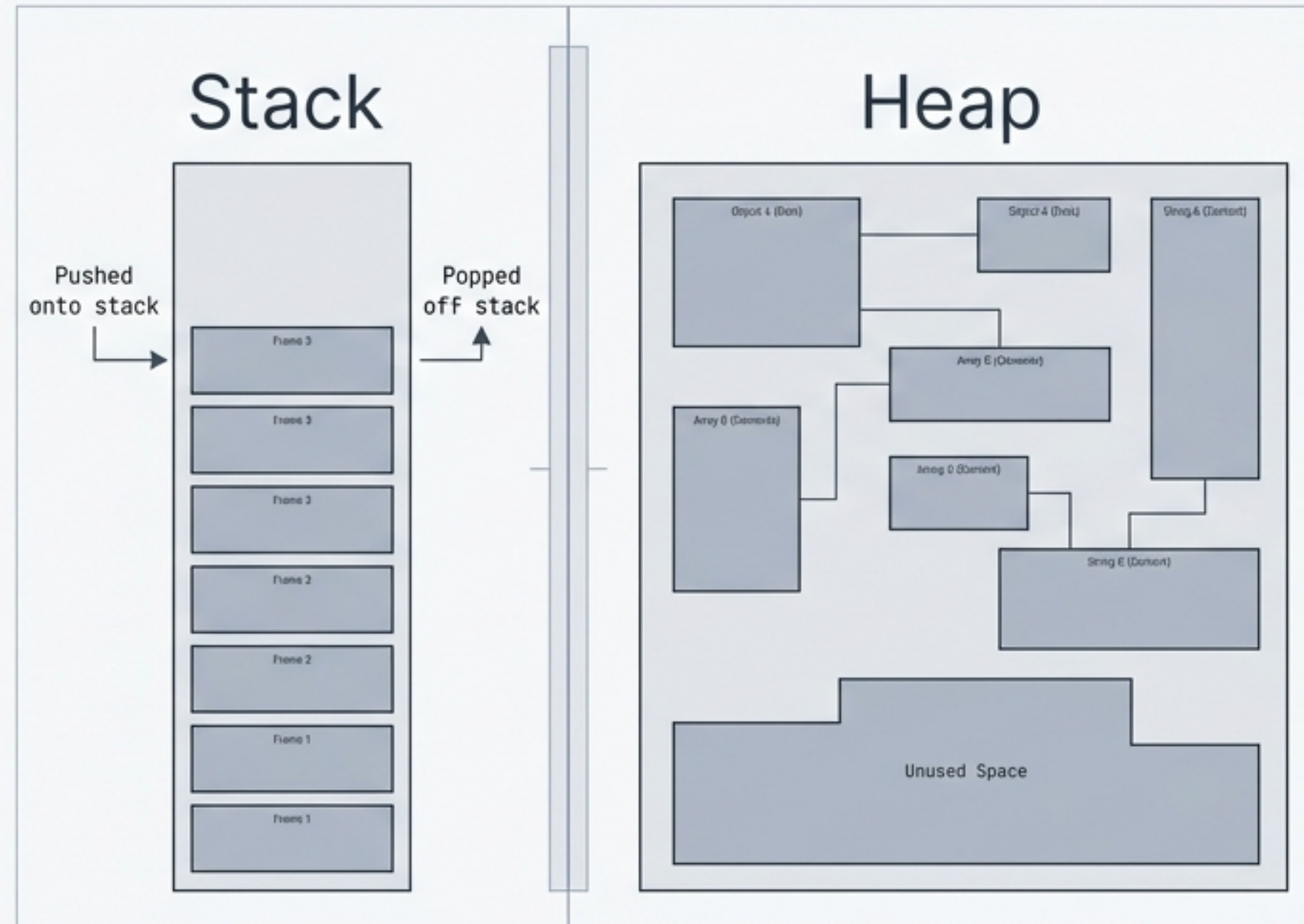
We assume primitives are simple and objects are complex. But what if one primitive breaks the rules for a very smart reason?

The Two Arenas of Memory: Stack and Heap

Before we dive into code, let's understand the two places our data lives.

The Stack

Highly **organized** and **fast**. Manages static data, like variable pointers (references) and primitive values. Think of it as a temporary scratchpad for function calls.



The Heap

A large, **less organized** region of memory. Stores the actual data for **complex objects** (like arrays and objects) and, as we'll see, strings.

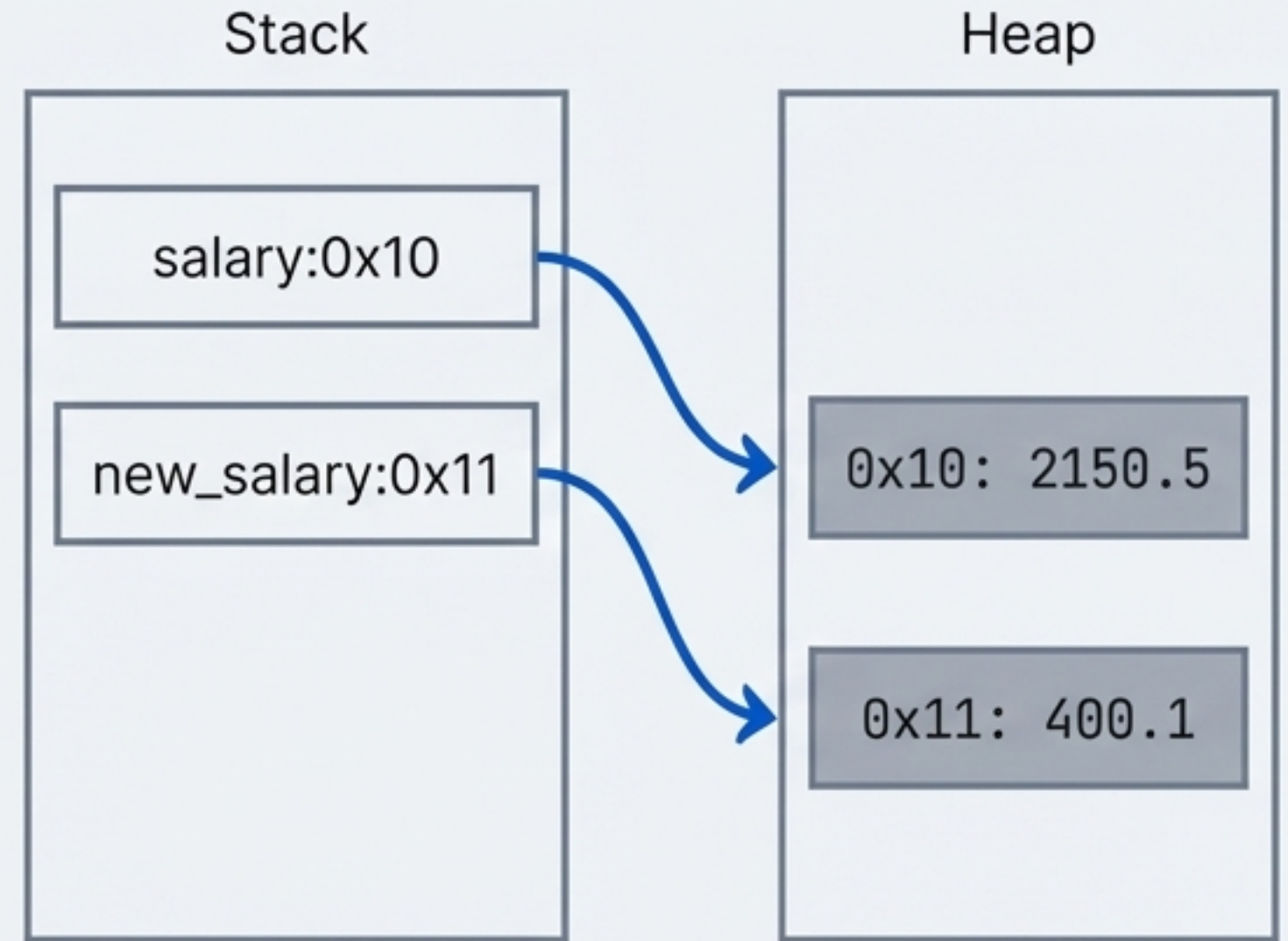
The Rule for Primitives: Copy by Value

When you assign a primitive like a number to a new variable, the system creates a distinct copy of that value in memory. The variables are completely independent.

```
let salary = 2150.5;  
let new_salary = salary;
```

```
u001b[48;5;214mnew_salary = 400.1;
```

```
console.log(salary);    // > 2150.5  
console.log(new_salary); // > 400.1
```



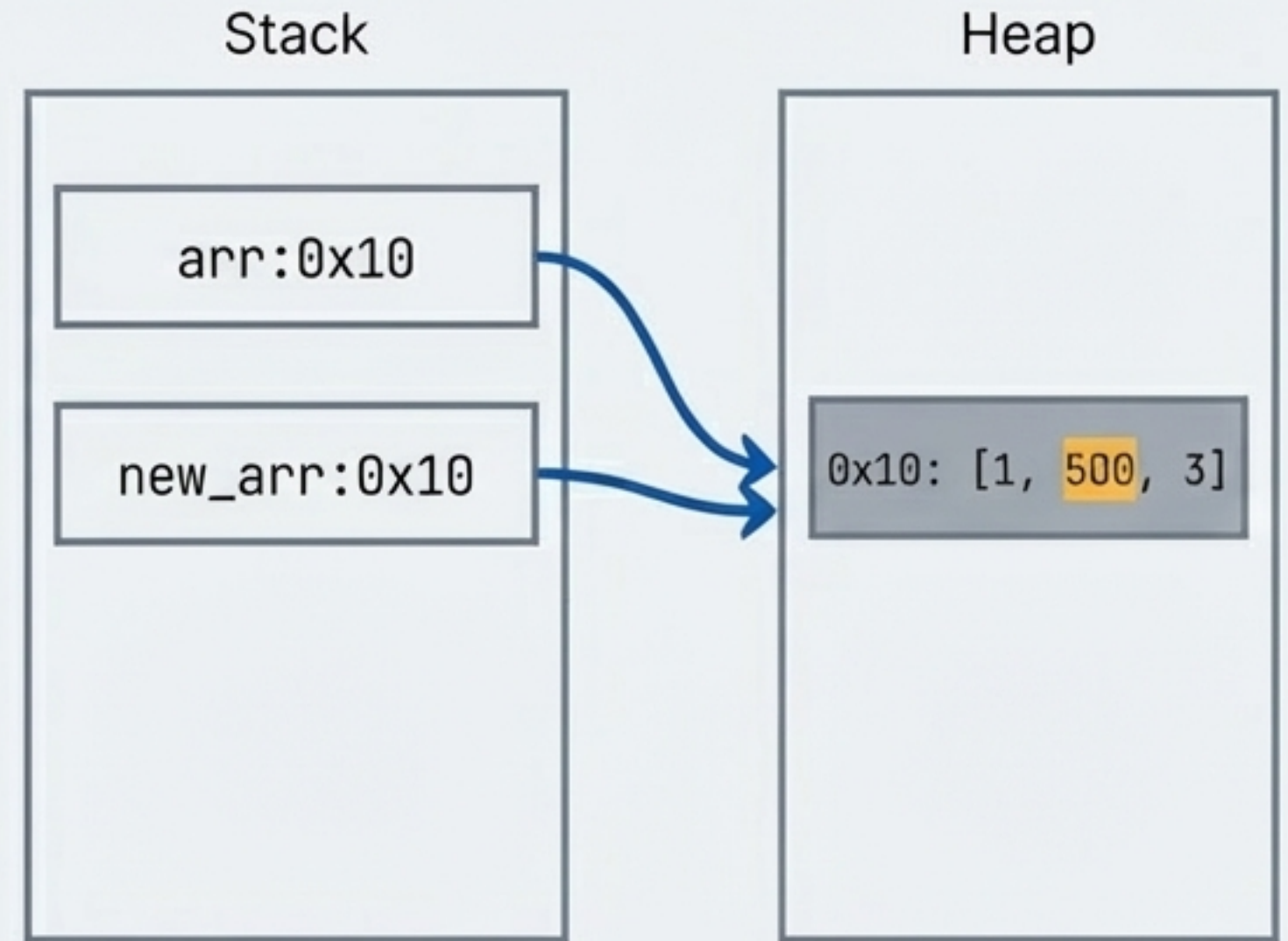
The Rule for Objects: Copy by Reference

When you assign an object or array, you're not copying the data. You're copying the memory address (the reference). Both variables point to the exact same object in the Heap.

```
let arr = [1, 2, 3];  
let new_arr = arr;
```

```
new_arr[1] = 500; // Modify through new_arr
```

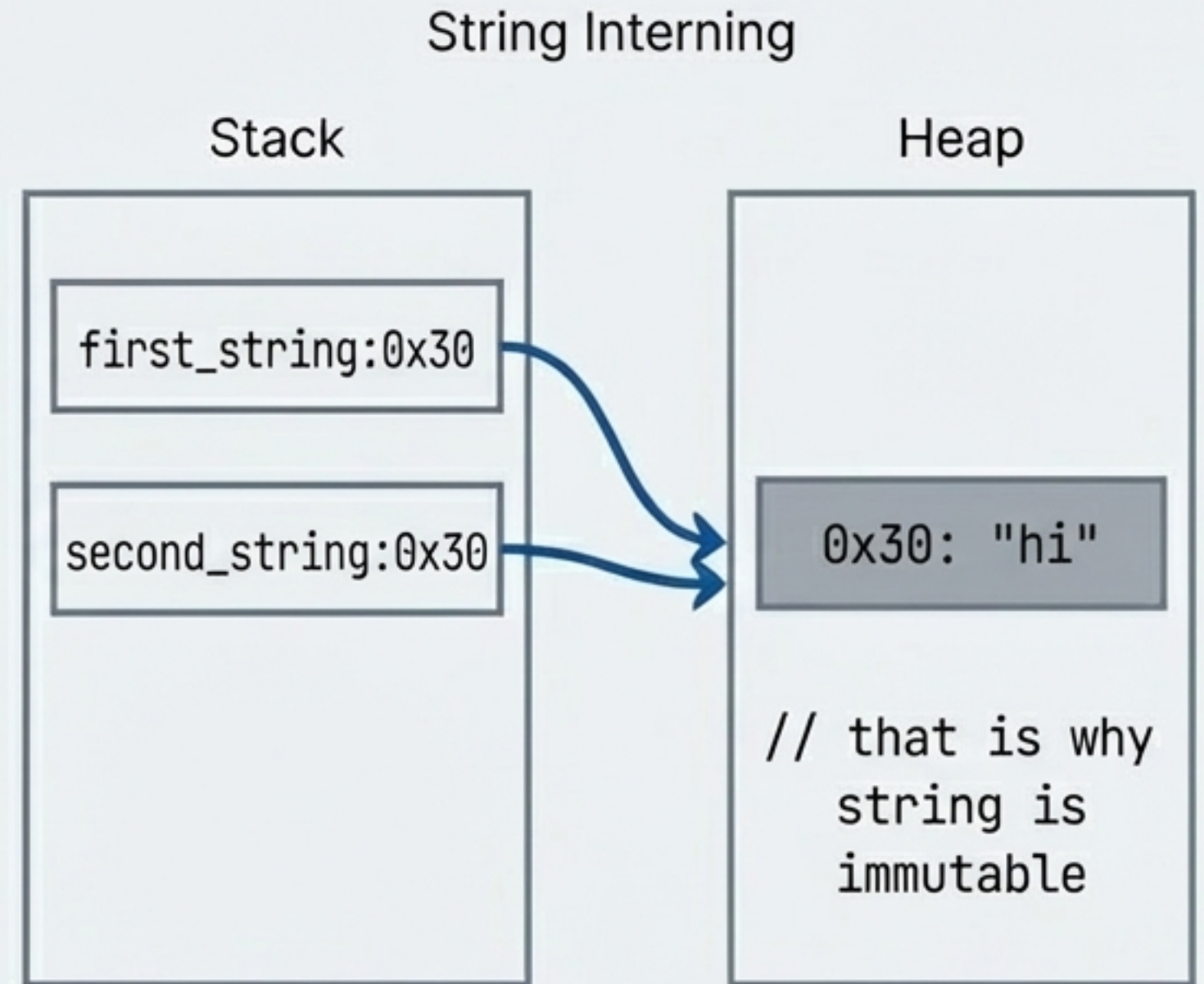
```
console.log(arr);      // > [1, 500, 3]  
console.log(new_arr);  // > [1, 500, 3]
```



So, Strings Behave Like Other Primitives... Right?

Logically, we'd expect two identical strings to be copied by value, each getting its own memory address. But the JavaScript engine is more clever than that.

```
let first_string = "hi";  
let second_string = "hi";
```

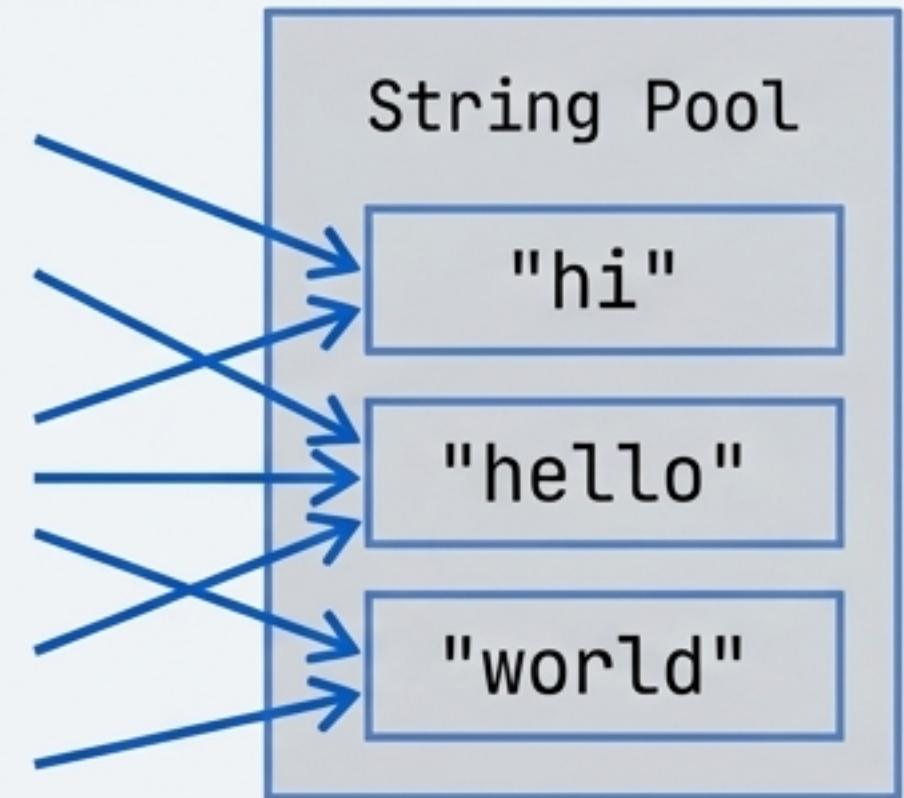


The Answer: A Memory Optimization Called String Interning

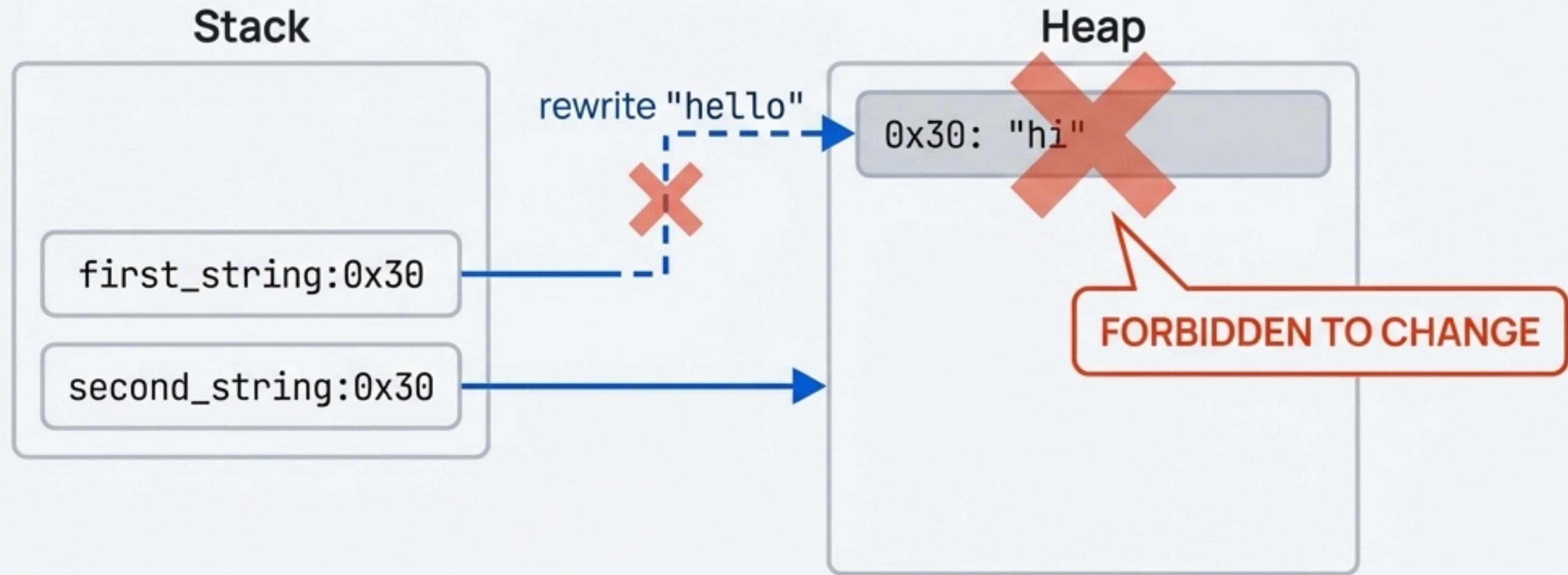
Node.js uses a method called **string interning**. Instead of storing duplicate strings, the engine keeps only **one one copy of each distinct string value** in memory.

- If it exists, the variable simply points to that existing address.
- If not, a new string is created in the pool, and the variable points to it.

This simple mechanism has profound consequences.



Interning Is the Reason Strings Must Be Immutable



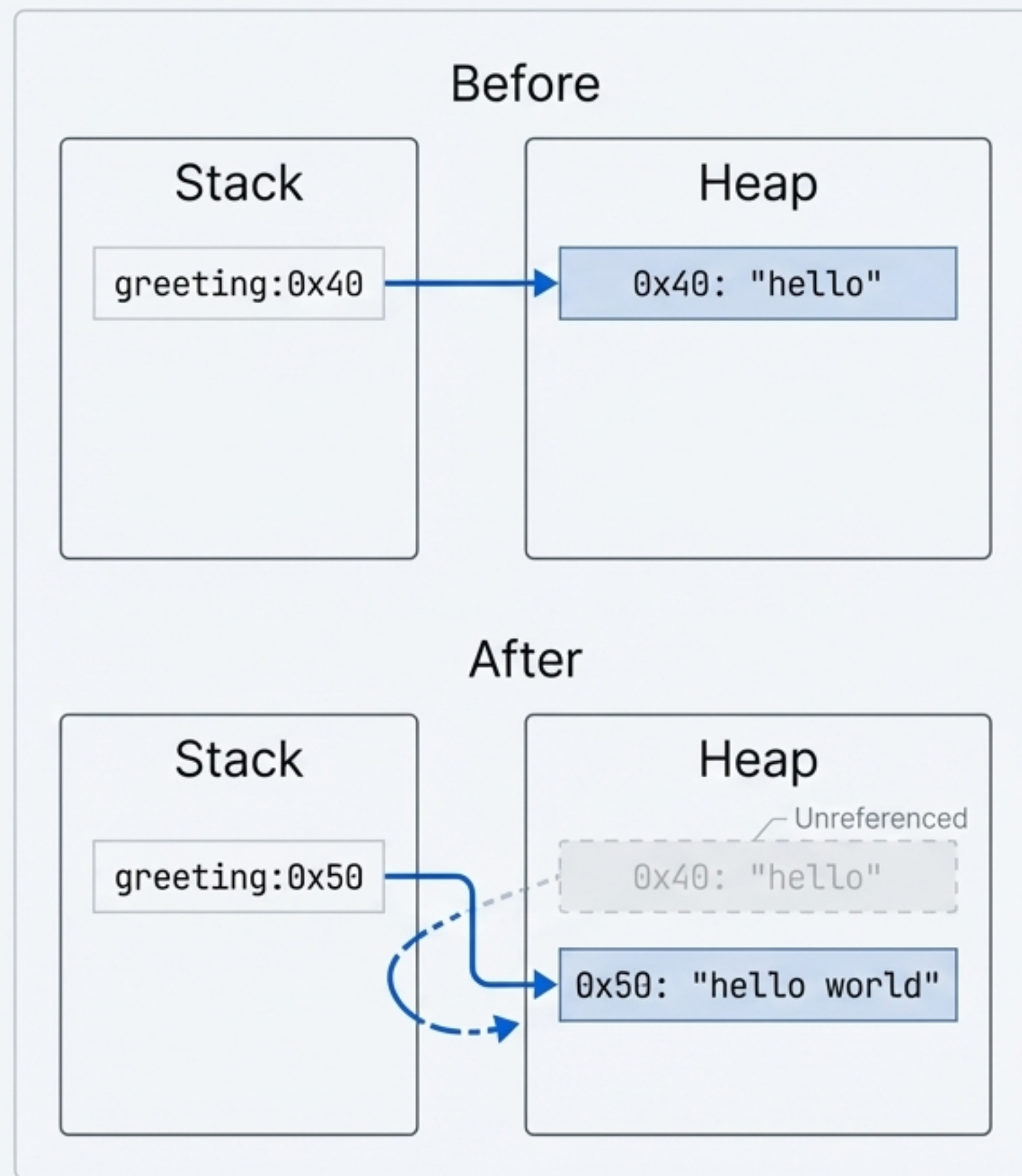
Imagine if `first_string` could change the value at the shared address `0x30` from `'hi'` to `'hello'`.

- This would instantly and unexpectedly change the value of `second_string` as well, along with every other variable in the application that happened to point to that interned string.
- To prevent this chaos, the language makes the value at that address **immutable**. It cannot be changed once created.

'Changing' a String Means Creating a New One

Since the original string data cannot be altered, any operation that seems to modify a string actually creates a brand new string at a new memory address. The variable's pointer is then updated to this new address. The original string remains untouched in the Heap (and may be garbage collected if no other variables point to it).

```
let greeting = "hello";  
  
// This doesn't change "hello".  
// It creates "hello world" at a new address.  
greeting = greeting + " world";
```

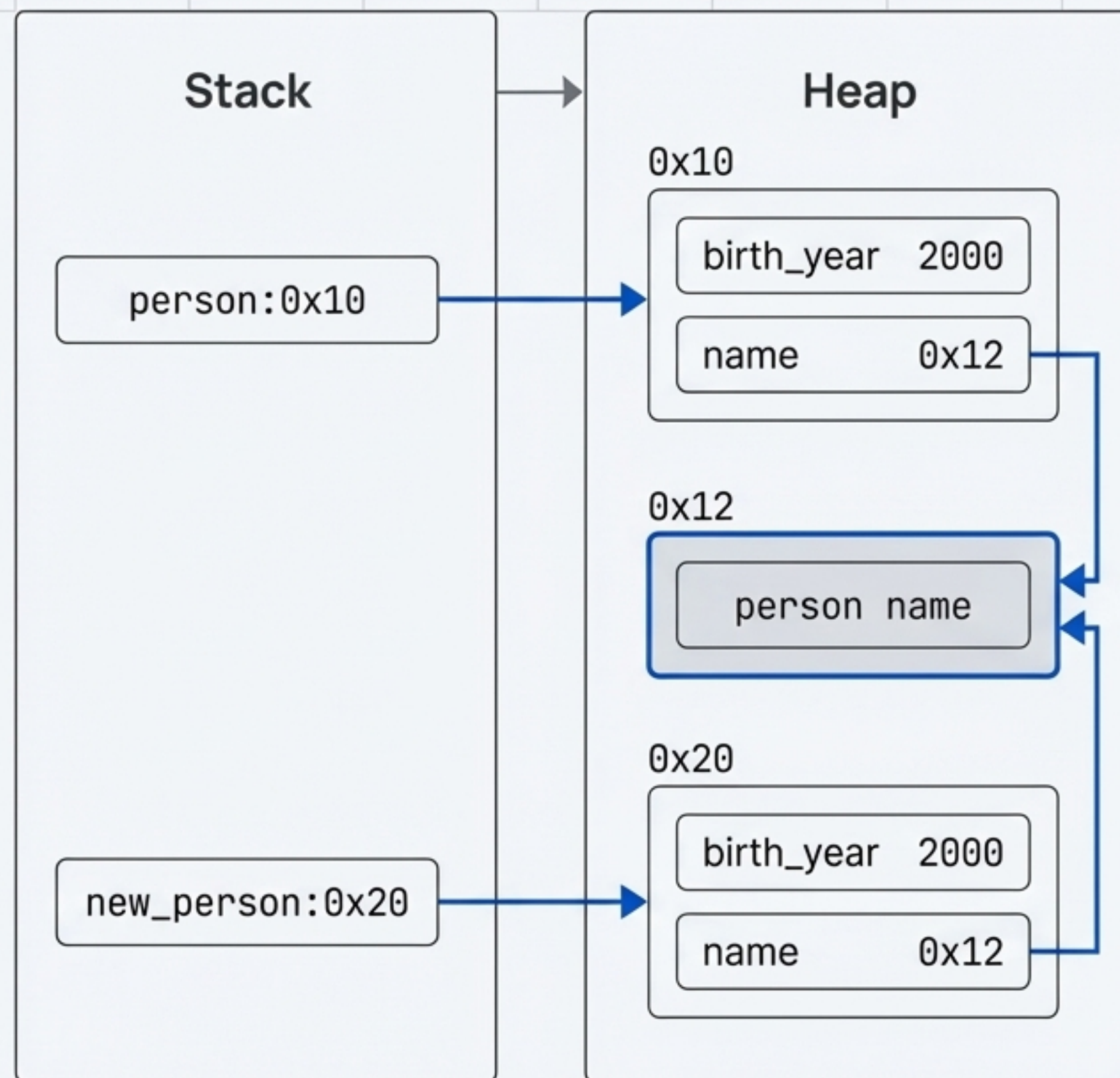


Interning in Action: Optimizing Object Copies

String interning provides benefits even inside other data structures. When you create a shallow copy of an object using the spread operator (...), a new object is created at a new address. However, the string properties inside it don't get duplicated; they simply point back to the original, interned string address in the Heap.

```
let person = {  
  birth_year: 2000,  
  name: "person name"  
};
```

```
let new_person = { ...person };
```



An Analogy: The Global Key Registry

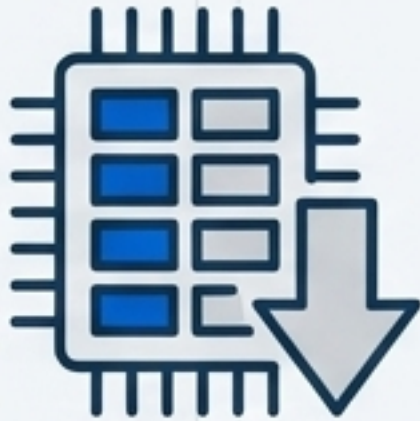
Think of **string interning** like a **Global Key Registry**. If ten people in an apartment building need a key to the same front door, the registry doesn't manufacture ten identical physical keys. Instead, it gives everyone the **same digital access code**.

Because the door (the string value) is "immutable" and never moves, everyone can safely share that single without ever needing a unique physical copy.



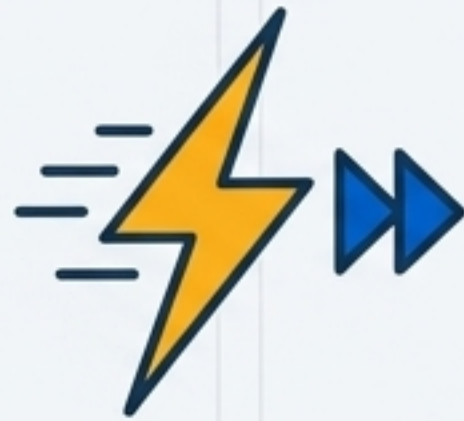
The Three Core Benefits of String Interning

This underlying mechanism provides tangible advantages in performance and memory management.



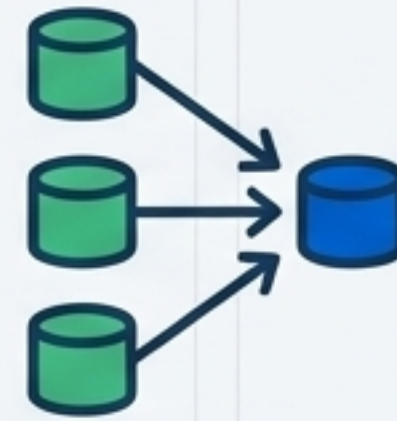
Memory Efficiency

Massively reduces the application's memory footprint by avoiding the storage of countless duplicate strings in the Heap.



Comparison Speed

Comparing two strings for equality (`str1 == str2`) can be as fast as comparing two integers. The engine just checks if they **point to the same memory address**, avoiding a slow character-by-character comparison.



Consistency

Guarantees that identical string literals used anywhere in the application are, in fact, the exact same object in memory.

From Memory Rules to Clever Optimization

Key Takeaways

1. **Primitives like numbers are copied by value.** Each variable gets its own unique copy of the data.
2. **Objects are copied by reference.** Multiple variables can point to, and modify, the very same object.
3. **Strings are the special primitive.** They are **interned**, meaning identical string values share a single memory address, just like an object reference.
4. **Interning necessitates immutability.** This ensures code predictability and enables powerful optimizations for memory and speed.